

Praktikum „Bildverarbeitung“ (WS 2025/26 – Dr.-Ing. Lorenz Fieten) – 2. Aufgabe

Eine weit verbreitete Aufgabe der Bildverarbeitung besteht darin, in Bildern Objekte (den sog. Vordergrund) vom Hintergrund zu trennen und als Ergebnis ein Binärbild auszugeben. Im einfachsten Fall basiert die Binarisierung auf einem globalen Schwellwert. Dieser kann je nach Anwendung interaktiv oder automatisiert an den Bildinhalt angepasst werden.

Aus der Vorlesung kennen Sie das sog. Otsu-Verfahren zur automatischen Bestimmung eines globalen Schwellwerts für die Binarisierung, welcher ein statistisches Optimalitätskriterium erfüllt. Dieses verhältnismäßig simple Verfahren kann in einer Vielzahl unterschiedlicher Anwendungen erfolgreich auf Grauwertbilder angewandt werden. Besonders gute Ergebnisse liefert das Otsu-Verfahren erfahrungsgemäß bei bimodalen Grauwertverteilungen mit ähnlich vielen hellen und dunklen Pixeln. Diese Bedingungen sind allerdings in der Praxis häufig nicht oder nur teilweise gegeben. Weiterhin ist zu berücksichtigen, dass das Verfahren nicht direkt anwendbar auf Mehrkanalbilder ist. Es kann jedoch auf einzelne Kanäle angewandt werden, wobei ein Kanal – wie z. B. bei dem von OpenCV unterstützten YCrCb-Format – auch Bild-Informationen über die Helligkeit beinhalten kann.

Im Rahmen dieser Aufgabe sollen wesentliche Aspekte der Binarisierung von Farbbildern auf der Basis interaktiv oder automatisiert bestimmter Schwellwerte mittels OpenCVs beleuchtet werden.

Dabei sollen insbesondere die folgenden Lernziele erreicht werden:

- Sie sind in der Lage, ein Farbbild zu laden und relevante Informationen zum Bildformat zusammen mit Statistiken zum Bildinhalt zu ermitteln.
- Sie sind in der Lage, zwischen verschiedenen Farbformaten zu konvertieren und für die jeweilige Darstellung die Kanäle eines Farbbilds zu extrahieren.
- Sie sind in der Lage, Grauwerte wie bei der aus der Vorlesung bekannten Grauwertspreizung zu transformieren.
- Sie sind in der Lage, eine interaktive, auf einem globalen Schwellwert basierende Binarisierung inkl. einer auf dem sog. Blending basierenden Visualisierung zu realisieren.
- Sie können beurteilen, ob eine auf einem globalen Schwellwert basierende Binarisierung für einen speziellen Anwendungsfall prinzipiell brauchbare Ergebnisse liefern kann.
- Sie sind in der Lage, ein Grauwert-Histogramm zu einem Kanal eines Farbbilds zu berechnen, zu visualisieren und zu interpretieren.
- Sie sind in der Lage, Gütekriterien für die Binarisierung wie das üblicherweise mit dem Otsu-Verfahren verbundene Gütekriterium zu berechnen, zu visualisieren und zu interpretieren.
- Sie können die Eignung verschiedener Kanäle von Farbbildern bzgl. der Binarisierung in einem speziellen Anwendungsfall beurteilen.
- Sie können den Nutzen verschiedener Visualisierungstechniken hinsichtlich einer intuitiven interaktiven Bestimmung von globalen Schwellwerten bewerten.
- Sie haben praktische Erfahrung darin, automatische Verfahren kritisch zu hinterfragen und ihre Funktionsweise zu analysieren, anstatt sie als „black box“ zu betrachten.

1. Öffnen Sie mittels ImageJ (File -> Open Sample...) die Datei hela-cells.tif.
2. Speichern Sie diese Datei unter demselben Namen an einem Ort Ihrer Wahl ab.
3. Legen Sie ein neues Projekt an (Konsolen-App): z. B. „ConsoleApplication_Task2“.
4. Erstellen Sie ein lokales Git-Repository.
5. Setzen Sie die Projekt-Eigenschaften so, dass OpenCV benutzt werden kann.
 - a. Für alle Konfigurationen unter C/C++ -> Allgemein
 „Zusätzliche Includeverzeichnisse“: C:\opencv\build\include
 - b. Für alle Konfigurationen unter Linker -> Allgemein
 „Zusätzliche Bibliotheksverzeichnisse“: C:\opencv\build\x64\vc16\lib
 - c. Für Debug-Konfiguration unter Linker -> Eingabe
 „Zusätzliche Abhängigkeiten“: opencv_world4120d.lib
 - d. Für Release-Konfiguration unter Linker -> Eingabe
 „Zusätzliche Abhängigkeiten“: opencv_world4120.lib eintragen
6. Fügen Sie oberhalb von `int main()` `#include „opencv2/opencv.hpp“` ein.
7. Legen Sie innerhalb von `int main()` ein Objekt vom Typ `cv::Mat` an: z. B. `inputImage`.
8. Sobald sich der Code kompilieren lässt, committen Sie Ihren Code.
9. Laden Sie in `int main()` die o. g. Datei derart, dass die `cv::Mat` ein Bild im BGR-Format enthält.
 Zur Orientierung: <https://opencv.org/blog/read-display-and-write-an-image-using-opencv/>
10. Unterdrücken Sie ggf. störende Log-Meldungen auf der Konsole.
 Hinweis: Werfen Sie einen Blick in den Namespace `cv::utils::logging`.
11. Geben Sie wichtige Eigenschaften der `cv::Mat` auf der Konsole aus. Schreiben Sie hierfür eine kleine Funktion, die auf der Konsole die folgenden Infos ausgibt:
 - Image specification: ... (Anm.: Hier sollte insb. das Farbsystem angegeben werden)
 - Number of channels: ...
 - Bit depth: ...
 - Image width: ...
 - Image height: ...
 - Minimum value: ... (... occurrences)
 - Maximum value: ... (... occurrences)
 Überlegen Sie, welche Angaben Sie der `cv::Mat` entnehmen können und welche Angaben Sie selber (sinnvollerweise als Parameter) zur Verfügung stellen müssten. Versuchen Sie, durch den konsequenten Einsatz von OpenCV-Funktionen wie `cv::split(...)`, `cv::minMaxLoc(...)` und `cv::countNonZero(...)` die Anzahl der benötigten Code-Zeilen klein zu halten.
12. Zeigen Sie das geladene (48-bit) Bild in einem Fenster an.

13. Transformieren Sie die Pixel-Werte-Tripel in Analogie zu der aus der Vorlesung bekannten Grauwertspreizung so, dass 255 als größter Wert resultiert, und speichern Sie das Ergebnis in einer 3-kanaligen `cv::Mat`, in der jeder Kanal 8-Bit-Daten enthält. Benutzen Sie hierfür die Funktion `cv::normalize(...)`.
14. Geben Sie die Eigenschaften der resultierenden `cv::Mat` auf der Konsole aus. Kontrollieren Sie, ob der größte Wert nun – wie gewünscht – 255 ist.
15. Zeigen Sie das resultierende Farbbild in einem Fenster an.
16. Zerlegen Sie das resultierende Farbbild in seine 3 Kanäle, d. h. erzeugen Sie 3 `cv::Mats` aus dem resultierenden Farbbild.
17. Geben Sie die Eigenschaften dieser 3 `cv::Mats` auf der Konsole aus. Welche größten Werte werden nun angezeigt? Was folgt daraus bzgl. der Funktionalität von `cv::normalize(...)`?
18. Zerlegen Sie nun das ursprüngliche (48-bit) Bild in seine 3 Kanäle, d. h. erzeugen Sie 3 `cv::Mats` aus dem ursprünglichen Farbbild.
19. Geben Sie die Eigenschaften dieser 3 `cv::Mats` auf der Konsole aus. Welche größten Werte werden nun angezeigt?
20. Transformieren Sie für jeden Kanal individuell die Pixel-Werte in Analogie zu der aus der Vorlesung bekannten Grauwertspreizung so, dass 255 als größter Wert resultiert, und speichern Sie das Ergebnis jeweils in einer 1-kanaligen `cv::Mat`, in der jeder Kanal 8-Bit-Daten enthält. Benutzen Sie hierfür die Funktion `cv::normalize(...)`.
21. Geben Sie die Eigenschaften dieser 3 `cv::Mats` auf der Konsole aus. Welche größten Werte werden nun angezeigt?
22. Erklären Sie mögliche Unterschiede zu den zuvor erhaltenen `cv::Mats`. Für welche Zwecke erscheinen die zuvor erhaltenen `cv::Mats` vorteilhaft bzw. nachteilig? Begründen Sie Ihre Antwort. Benutzen Sie für die weiteren Unteraufgaben bitte die zuletzt erhaltenen `cv::Mats`.
23. Zeigen Sie die 3 zuletzt erhaltenen `cv::Mats` in 3 Fenstern an. Verwenden Sie hierfür die Namen „8-bit B channel“, „8-bit G channel“ und „8-bit R channel“.
24. Fügen Sie jedem der Fenster 2 Trackbars hinzu, einen für einen Schwellwert (im Bereich 0...255) und einen für einen prozentualen Transparenzgrad (0...100), z. B. mit `cv::createTrackbar("threshold B", " 8-bit B channel ", nullptr, 255, onThresholdBlueTrackbar);`
25. Verknüpfen Sie diese Trackbars mit den globalen Variablen `g_thresholdB`, `g_thresholdG`, `g_thresholdR`, `transparencyB`, `g_transparencyG`, `g_transparencyR` (vom Typ `int`), und geben Sie geänderte Werte dieser globalen Variablen auf der Konsole aus.
26. Schreiben Sie eine Funktion, die aus einem Einkanalbild ein Binärbild derart erzeugt, dass alle Pixel bis zu einem vorgegebenen Schwellwert schwarz sind. (Hinweis: Beachten Sie, was die Anwendung des `>`-Operators auf eine `cv::Mat` bewirkt.)

27. Modifizieren Sie die Funktion derart, dass alle Pixel oberhalb des Schwellwerts eine vorgegebene Farbe (übergeben als `cv::Scalar`) erhalten und alle Pixel bis zum Schwellwert einen Grauwert erhalten, der bei Betrachtung des HSV-Farbsystems etwa als gleich hell wahrgenommen würde wie die vorgegebene Farbe. Wie lässt sich dieser Grauwert aus den Komponenten des `cv::Scalars` berechnen?
28. Schreiben Sie eine Funktion, die ein Überlagerungsbild aus dem zuletzt berechneten Bild und dem zugrundeliegenden Einkanalbild (z. B. B-Kanal) berechnet. Benutzen Sie hierfür die Funktion `cv::addWeighted(...)`. Die gewünschte Funktion soll eine `cv::Mat` (für das Einkanalbild), einen Schwellwert, eine zum Einkanalbild passende Farbe und einen Transparenzgrad als Argumente erhalten und eine `cv::Mat` ausgeben.
29. Integrieren Sie die entwickelte Funktionalität so in die Konsolen-Anwendung, dass in den Fenstern „8-bit B channel“, „8-bit G channel“ und „8-bit R channel“ die zu den aktuell per Trackbars eingestellten Parametern (Schwellwerte und Transparenzgrade) passenden Überlagerungsbilder angezeigt werden.
30. Benutzen Sie die bisher entwickelte Funktionalität, um herauszufinden, inwiefern die im Farbbild bläulich dargestellten Zellkerne mittels eines interaktiv vorgegebenen globalen Schwellwerts zufriedenstellend segmentiert werden können. Zu welcher Schlussfolgerung kommen Sie im konkreten Anwendungsfall (d. h. bezogen auf `hela-cells.tif`)?
31. Schreiben Sie eine Klasse `OtsuThresholdProvider`, welche zu einem gewöhnlichen absoluten Histogramm den optimalen Schwellwert berechnet. Die Klasse soll u. a. Member-Variablen für die Ihnen bekannten Histogramme sowie für die Werte des Gütekriteriums (Quotient aus schwelwertspezifischer Zwischen-Klassen-Varianz und Gesamt-Varianz) für alle potentiellen Schwellwerte haben.
Es können z. B. die folgenden Member-Variablen benutzt werden:
- ```
size_t m_numPixels;
std::vector<size_t> m_ordinaryAbsoluteHistogram;
std::vector<double> m_ordinaryRelativeHistogram;
std::vector<size_t> m_cumulativeAbsoluteHistogram;
std::vector<double> m_cumulativeRelativeHistogram;
double m_mean;
double m_variance;
std::vector<double> m_criterionMeasures;
int m_otsuThreshold;
```
- Die Werte aller Member-Variablen können bereits im Konstruktor berechnet werden.  
Hinweise zu einer möglichen Implementierung finden Sie im folgenden Open-Access-Artikel:  
*Juan Pablo Balarini, Sergio Nesmachnow, A C++ Implementation of Otsu's Image Segmentation Method, Image Processing On Line, 6 (2016), pp. 155-164*
32. Erweitern Sie die Funktionen `onThresholdBlueTrackbar(...)`, `onThresholdGreenTrackbar(...)` und `onThresholdRedTrackbar(...)` derart, dass bei Änderungen der Schwellwerte neben einem aktualisierten Schwellwert auch der Wert des (Otsu-) Gütekriteriums angezeigt wird. (Hinweis: Dieser liegt im Intervall 0...1.)

33. Schreiben Sie eine Funktion, welche ein „Liniendiagramm“ in Form einer `cv::Mat` ausgibt und der Visualisierung der relativen Histogramme und der Werte des Gütekriteriums dient. Sie können hierfür Code aus diesem Tutorial an die Aufgabenstellung anpassen:  
[https://docs.opencv.org/3.4/d8/dbc/tutorial\\_histogram\\_calculation.html](https://docs.opencv.org/3.4/d8/dbc/tutorial_histogram_calculation.html)
34. Zeigen Sie jeweils 1 Liniendiagramm in den folgenden 9 Fenstern an:  
 "criterion measures for B channel"  
 "criterion measures for G channel"  
 "criterion measures for R channel"  
 "ordinary relative histogram for B channel"  
 "ordinary relative histogram for G channel"  
 "ordinary relative histogram for R channel"  
 "cumulative relative histogram for B channel"  
 "cumulative relative histogram for G channel"  
 "cumulative relative histogram for R channel"
35. Erweitern Sie die Funktionalität zur Liniendiagramm-Erstellung derart, dass nur Werte für (x-) Positionen oberhalb eines übergebenen Schwellwerts (default: 0) in der per `cv::Scalar` übergebenen Farbe dargestellt werden, Werte für (x-) Positionen bis zu diesem Schwellwert jedoch in einem Grauton mit ähnlicher Helligkeitswahrnehmung (gemäß HSV-Farbsystem).
36. Aktualisieren Sie die Anzeige der Liniendiagramme gleichzeitig mit der Anzeige der Überlagerungsbilder, damit der jeweils aktuelle Schwellwert darin ersichtlich ist
37. Erweitern Sie die Funktionalität um ein Abspeichern der in den Fenstern angezeigten `cv::Mats` per Drücken der Taste ‚s‘. Dabei sollten die relevanten Schwellwerte und ggf. Transparenzgrade in den Dateinamen enthalten sein.
38. Testen und bewerten Sie die Gesamtfunktionalität. Können mit dem Otsu-Schwellwert akzeptable Ergebnisse erhalten werden? Wie könnte die Qualität ggf. erhöht werden?
39. Konvertieren Sie testweise nach dem Öffnen der Datei `hela-cells.tif` das Farbbild in die Farbsysteme YCrCb und Lab. Beachten Sie bitte die Hinweise aus der OpenCV-Doku:  
[https://docs.opencv.org/4.12.0/de/d25/imgproc\\_color\\_conversions.html](https://docs.opencv.org/4.12.0/de/d25/imgproc_color_conversions.html)
40. Optional: Ermitteln Sie den Fehler, der sich aus Hin- und anschließender Rückkonvertierung ergibt.
41. Zerlegen Sie das Farbbild jeweils (also für YCrCb und Lab) in 3 Kanäle, transformieren Sie für jeden Kanal individuell die Pixel-Werte in Analogie zu der aus der Vorlesung bekannten Grauwertspreizung so, dass 255 als größter Wert resultiert, und speichern Sie das Ergebnis jeweils in einer 1-kanaligen `cv::Mat`, in der jeder Kanal 8-Bit-Daten enthält.
42. Ersetzen Sie testweise die zuvor verwendeten B-/G-/R-Kanäle durch die neu erhaltenen Kanäle und überprüfen Sie, wie sich dies auf die Segmentierungsergebnisse auswirkt. Haben Sie eine Erklärung für Ihre Beobachtungen?

*Melden Sie sich gerne bei Unklarheiten.*